Neural Network and ANFIS based auto-adaptive MPC for path tracking in autonomous vehicles

Yassine Kebbati
IBISC-EA4526
Université Paris-Saclay
Evry, France
yassine.kebbati@univ-evry.fr

Naima Ait-Oufroukh
IBISC-EA4526
Université Paris-Saclay
Evry, France
naima.aitoufroukh@univ-evry.fr

Vincent Vigneron
IBISC-EA4526
Université Paris-Saclay
Evry, France
vincent.vigneron@univ-evry.fr

Dalil Ichalal
IBISC-EA4526
Université Paris-Saclay
Evry, France
dalil.ichalal@univ-evry.fr

2021 IEEE International Conference on Networking, Sensing and Control (ICNSC) | 978-1-6654-4048-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICNSC52481.2021.9702227

***Abstract*— Self-driving cars operate in constantly changing environments and are exposed to a variety of uncertainties and disturbances. These factors render classical controllers ineffective, especially for the lateral control in such vehicles. Therefore, an adaptive MPC controller is designed in this paper for the path tracking task, the developed controller is tuned by an improved particle swarm optimization algorithm. Furthermore, online parameter adaption is performed using Neural Networks and ANFIS. The designed controller showed promising results and adaptation capability against the standard MPC in a triple lane change scenario and a general trajectory test.**

***Index Terms*— Autonomous Vehicles, Optimization, Model Predictive Control, Adaptive Control, Neural Networks.**

I. Introduction

Autonomous driving is one of the top technologies that can radically change people's life, owing to the fact that self-driving cars can significantly reduce road accidents, alleviate traffic congestion and mitigate energy consumption and air pollution. As a consequence, researchers are racing towards building fully autonomous driving systems. One of the main components of such systems is the control module which handles lateral and longitudinal control tasks. Lateral control is a primordial part to achieve autonomous driving, which has seen extensive research over the last decades. Although there exist several simple controllers in the literature that can easily control the vehicle lateral dynamics, most of them fail to cope with external disturbances and preserve safety constraints.

Model predictive control can handle constraints systematically setting itself as a promising tool for such tasks [1], [2]. On the other hand, adaptive control has shown its ability to handle model uncertainties, disturbances and to deal with varying parameters [3]–[6]. As the computational power increased recently, researchers aimed at integrating AI techniques to improve and adapt controllers to varying working conditions, disturbances and modeling uncertainties. These research works either use data to learn the models used in the controller design, or learn the design itself by adapting the control algorithm. For instance, Alcala *et al*. [7] developed an MPC controller for autonomous vehicles where they used an ANFIS approach to learn the vehicle dynamics model in the form of a polytopic representation. Similarly, authors of [8] developed an adaptive MPC for lane keeping that can handle unknown steering offsets through a recursive model estimation algorithm. In the same fashion, Lin *et al*. [9] designed an adaptive MPC that handles changing working conditions. They used the recursive least square (RLS) algorithm to estimate the vehicle cornering stiffness and road adhesion coefficients to adapt the MPC prediction model. Brunner *et al*. [10] proposed a learning-based MPC for autonomous racing where racing data was used to learn a terminal cost and a safe set that guarantees recursive stability and improved performance over iterations. In a similar way, Kabzan *et al*. [11] proposed a learning-based MPC for autonomous racing. They used Gaussian process regression to improve the MPC prediction model using driving data after several racing laps. In [12], an MPC with low computation load has been proposed. The authors approximated the control signal with Laguerre functions and improved the tracking accuracy using exponential weight technique. Furthermore, authors of [13] considered the varying road conditions and small slip-angle assumptions as a measurable disturbance in the MPC design. They used the differential evolutionary algorithm to solve the problem.

Nonetheless, very few research papers worked on learning the control algorithm and most of them only consider constant longitudinal velocities. This paper proposes a new approach to learn and adapt the MPC algorithm to external disturbances and varying working conditions. Particularly, an improved PSO algorithm developed in [14] is used to tune and optimize several parameters of the MPC algorithm to ensure optimal control performance. The resulting optimal parameters are learned using neural networks and ANFIS systems for online control adaptation. Section II of this papers deals with vehicle modeling and MPC design, and section III exposes the optimization of MPC parameters. Section IV presents the controller adaptation using neural

networks and ANFIS approach. The proposed controller is evaluated and the results are reported and analysed in section V. Conclusions and perspectives for future work are given in section VI.

II. Vehicle Modeling and Controller Design

A. Vehicle Lateral Dynamics Model

Vehicle lateral control deals with the translation along the y-axis and the angular motion around the z-axis. The single track dynamic bicycle model (see Fig. 1) is used for the design of the MPC controller, this model is governed by the following equations:

$$\begin{cases} m(\ddot{y}+\dot{x}\dot{\phi}) & = 2F_{yf}+2F_{yr} \\ I_z\ddot{\phi} & = 2l_fF_{yf}-2l_rF_{yr} \end{cases} \quad (1)$$

where, m is the vehicle mass, x, y and ϕ represent the longitudinal position, lateral position and the vehicle heading angle. The longitudinal forces on the rear and front wheels are F_{yr} and F_{yf} respectively, $l_r \backslash l_f$ are the distances between the rear\front wheel axles and the vehicle's gravity center (CG). F_y results from the nonlinear tire-road interactions, which can be linearized under the assumption of small slip angles (typically no more than 5°). The linear tire model is given as follows:

$$\begin{cases} F_{yf} & = C_{yf}\alpha_f \\ F_{yr} & = C_{yr}\alpha_r \end{cases} \quad (2)$$

with $C_{yf} \backslash C_{yr}$ being the stiffness coefficients for front\rear wheels respectively, $a_f \backslash a_r$ are the respective lateral slip angles for the front\rear wheels which are given by:

$$\begin{cases} \alpha_f & = \delta_f - \gamma_f \\ \alpha_r & = \delta_r - \gamma_r \end{cases} \quad (3)$$

where $\delta_r \backslash \delta_f$ are the steering angles for the rear\front wheels, the rear wheel is not steerable ($\delta_r = 0$). The angles between the direction of the wheels and the longitudinal velocity are given as:

$$\begin{cases} \tan(\gamma_f) & = \frac{\dot{y}+l_f\dot{\phi}}{\dot{x}} \\ \tan(\gamma_r) & = \frac{\dot{y}-l_r\dot{\phi}}{\dot{x}} \end{cases} \quad (4)$$

Equation (5) transforms the lateral velocity from the body frame to the inertial frame:

$$\dot{Y} = \dot{x}\sin\phi + \dot{y}\cos\phi \quad (5)$$

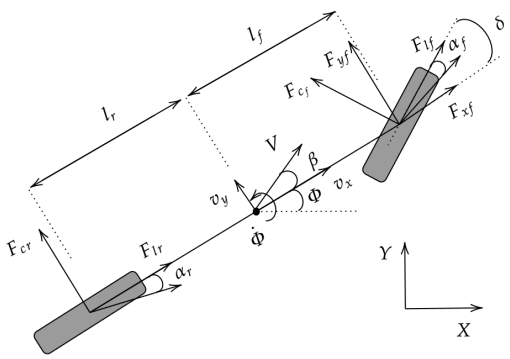

Fig. 1: 2-DOF Bicycle dynamic model.

The bicycle dynamic model is then obtained by replacing the above mentioned equations in (1):

$$\begin{cases} m(\ddot{y}+\dot{x}\dot{\phi}) & = 2[C_{yf}(\delta_f - \frac{\dot{y}+l_f\dot{\phi}}{\dot{x}}) + C_{yr}\frac{l_r\dot{\phi}-\dot{y}}{\dot{x}}] \\ I_z\ddot{\phi} & = 2[l_fC_{yf}(\delta_f - \frac{\dot{y}+l_f\dot{\phi}}{\dot{x}}) - l_rC_{yr}\frac{l_r\dot{\phi}-\dot{y}}{\dot{x}}] \end{cases} \quad (6)$$

The corresponding state space representation is as follows:

$$\begin{cases} \dot{x}_c & = A_cx_c + B_cu \\ y_c & = C_cx_c \end{cases} \quad (7)$$

with the state vector $x_c = [\dot{y}\ \phi\ \dot{\phi}\ y]^T$, the output $y_c = y$ and the input $u = \delta_f$. The state, control and output matrices are given by:

$$A_c = \begin{bmatrix} -2\frac{C_{yf}+C_{yr}}{m\dot{x}} & 0 & -\dot{x} - 2\frac{C_{yf}l_f - C_{yr}l_r}{m\dot{x}} & 0 \\ 0 & 0 & 1 & 0 \\ -2\frac{C_{yf}l_f - C_{yr}l_r}{I_z\dot{x}} & 0 & -2\frac{C_{yf}l_f^2 + C_{yr}l_r^2}{I_z\dot{x}} & 0 \\ 1 & \dot{x} & 0 & 0 \end{bmatrix},$$

$$B_c = \begin{bmatrix} 2\frac{C_{yf}}{m} \\ 0 \\ 2\frac{C_{yf}l_f}{I_z} \\ 0 \end{bmatrix}, \qquad C_c = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}.$$

B. MPC Design

The linear MPC, having a low computational burden, is considered in this paper. The principle of MPC is based on using the plant model to predict its response over a prediction horizon N_p, and generating a control sequence over a control horizon N_c. The control sequence, which is optimal and minimizes the tracking error, is obtained by solving a constrained convex optimization problem. Although the control sequence is optimized all along the control horizon N_c, only the first term is applied. The discretized form of model (7) is used as the prediction model of the MPC:

$$\begin{cases} x_{(k+1)} & = A_kx_{(k)} + Bu_{(k)} \\ y_{(k)} & = Cx_{(k)} \end{cases} \quad (8)$$

However, the state matrix A_k is updated at each iteration with the actual longitudinal velocity unlike most studies in the literature [8], [9], [11], [12]. This results in an adaptive prediction model that works for varying velocity profiles and not only constant velocities. The model is changed by adding an integrator and the output $y_{(k)}$ to the state vector to obtain an augmented model where the input becomes Δu_k:

$$\begin{cases} \tilde{x}_{(k+1)} & = \tilde{A}_k\tilde{x}_{(k)} + \tilde{B}\Delta\tilde{u}_{(k)} \\ \tilde{y}_{(k)} & = \tilde{C}\tilde{x}_{(k)} \end{cases} \quad (9)$$

The new state is $\tilde{x} = \begin{bmatrix}\Delta x_{(k)} & y_{(k)}\end{bmatrix}^T$, the new input is $\Delta\tilde{u}_{(k)}$ and the augmented system matrices become as the following:

$$\tilde{A} = \begin{bmatrix} A_k & o_m^T \\ CA_k & I_{q\times q} \end{bmatrix}, \ \tilde{B} = \begin{bmatrix} B \\ CB \end{bmatrix}, \ \tilde{C} = [o_m \ I_{q\times q}].$$

The term $o_m = \overbrace{[0\ 0...0]}^{n}$ is a vector of zeros and $I_{q\times q}$ is an identity matrix where n, m and q are the number of states, inputs and outputs respectively. The control sequence over the control horizon N_c is given by:

$$\Delta U = [\Delta u(k_i), \Delta u(k_i+1), ..., \Delta u(k_i+N_c-1)]$$

Model (9) predicts the plant behaviour over the prediction horizon N_p through the following equation:

$$Y = Fx_{(k_i)} + \Phi\Delta U \tag{10}$$

where matrices F and Φ are given by:

$$F = \begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^{N_p} \end{bmatrix}$$

$$\Phi = \begin{bmatrix} CB & 0 & \dots & 0 \\ CAB & CB & \dots & 0 \\ CA^2B & CAB & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & \dots & CA^{N_p-N_c}B \end{bmatrix}$$

The MPC problem is then defined as the following constrained optimization:

$$min\ J = (R_s - Y)^T Q(R_s - Y) + \Delta U^T R \Delta U \tag{11}$$

$$s.t : x(k+1) = Ax(k) + B\Delta u(k) \tag{12}$$

$$\Delta u_{min} \leq \Delta u \leq \Delta u_{max} \tag{13}$$

$$u_{min} \leq u \leq u_{max} \tag{14}$$

$$y_{min} \leq y \leq y_{max} \tag{15}$$

where Q and R are weighting matrices, R_s , Y and ΔU are the reference trajectory, the output vector and the control sequence respectively. The constraints are given in terms of ΔU as the following:

$$M\Delta U \leq \gamma \tag{16}$$

M is a combination of reformulation sub-matrices and γ groups the upper and lower bounds of the constraints. Thus, the MPC is formulated as the following quadratic programming (QP) problem:

$$\begin{cases} J = \frac{1}{2}x^T Ex + x^T K \\ Mx \leqslant \gamma \end{cases} \tag{17}$$

where x is the decision variable (Δu), E, K and M are compatible matrices and vectors with E being symmetric and positive definite. The solution of the given QP problem is based on the Hildreth's QP method [15].

III. Controller Optimization With Improved PSO

To optimize and tune the MPC parameters, we use an improved version of the PSO algorithm, which is a well known evolutionary algorithm in the literature [16], [17]. The standard algorithm is defined by:

$$\begin{cases} v_i(k+1) = & \omega v_i(k) + c_1 r_1(Pb_i(k) - x_i(k)) \\ & +c_2 r_2(Gb(k) - x_i(k)) \\ x_i(k+1) = & x_i(k) + v_i(k+1) \end{cases} \tag{18}$$

where v_i and p_i are the velocity and position of particle (i), which represents a solution to the optimization problem. ω, c_1 and c_2 are the inertia weight, the cognitive and the social accelerations respectively, and $r_{1,2} \in [0, 1]$ are random constants. Pb and Gb represent the best local and global positions respectively. ω and $c_{1,2}$ are constants in the classic algorithm, however, in the improved version of this work they are given by the following equations [14]:

$$\omega = \omega_{min} + \frac{\exp\left(\omega_{max} - \lambda_1(\omega_{max} + \omega_{min})\frac{g}{G}\right)}{\lambda_2} \tag{19}$$

$$\begin{cases} c_1(k+1) = c_1(k) + \alpha \\ c_2(k+1) = c_2(k) + \beta \\ \alpha = -\beta = 0.05 \quad for \quad \frac{g}{G} \leq 20\% \\ \alpha = -\beta = 0.02 \quad for \quad 20\% \leq \frac{g}{G} \leq 35\% \\ \alpha = -\beta = -0.035 \quad for \quad 35\% \leq \frac{g}{G} \leq 75\% \\ \alpha = -\beta = -0.0015 \quad for \quad \frac{g}{G} \geq 75\% \end{cases} \tag{20}$$

where g and G are the current generation and the maximum generation, ω_{min} and ω_{max} are minimum and maximum inertia weights and $\lambda_{1,2}$ are adjustable constants to ensure an exponential decrease from ω_{max} to ω_{min}. The advantage of this improved version over the standard one is that it enhances the overall search capabilities of the PSO algorithm [14], where the exponential decrease of ω accelerates the convergence towards the global best solution. On the other hand, increasing the cognitive acceleration c_1 enhances the exploration phase where particles tend to converge towards Pb and increasing c_2 enhances the exploitation phase where particles converge towards Gb and vice versa. The improved PSO algorithm is used to tune and optimize the MPC parameters which are N_p, N_c, Q and R. Fig. 2 shows the optimization approach where the MSE is used as the fitness function. Y_{ref} is the reference path, Y is the vehicle lateral position, $\dot{x}$ is the longitudinal velocity and X_{state} is the state vector. The PSO hyper-parameters used for the optimization are given in table (II), these parameters have been selected after evaluating the algorithm on typical benchmark problems like the sphere equation. To cover the majority of possible situations, the optimization is done for a variety of longitudinal speeds $\dot{x}(m/s) \in [3, 27]$ and lateral references $y_{ref}(m) \in [-15, 15]$. In addition, to account for external disturbances and consider different road conditions, the road adhesion coefficient is varied $\mu \in [0.5, 0.9]$, and the lateral wind is also included $v_w(m/s) \in [-30, 30]$. The result is a small data-set with optimal MPC parameters.

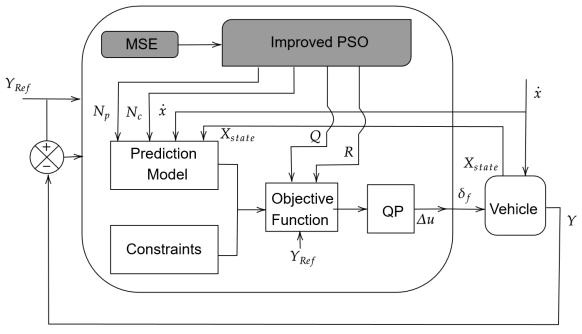

Fig. 2: Schematic diagram of the optimization approach.

TABLE I: PSO hyper-parameters.

Parameter	Interpretation	Value
N	Number of generations	15
N_{Pop}	Number of particles	20
ω_{max}	Maximum inertia weight	0.99
ω_{min}	Minimum inertia weight	0.1
c_{1i}	Initial cognitive acceleration coefficient	2
c_{2i}	Initial social acceleration coefficient	2
λ_1	Constant	30
λ_2	Constant	3

IV. MPC Parameters Adaptation

A. MPC adaptation with neural networks

After the optimisation phase which is done offline, a data-set of optimal MPC parameters is generated. In the first approach, four feedforward neural networks are used to learn the optimal MPC parameters for the sake of online adaptation. The proposed neural networks consist of four layers; the first layer contains the same four inputs, the first hidden layer consists of 16 neurons, the second hidden layer contains 8 hidden neurons and the output layer corresponds to one of the four MPC parameters: N_p, N_c, Q or R as shown in Fig. 3. The Sigmoid activation function is used for the hidden layers and the relu function is used for the output layer since this is a regression problem for positive values only. The loss function used for the back-propagation is the MSE along with the gradient decent with momentum.

The forward pass in the neural network is governed by equations (21), (22) and (23) respectively:

$$H_j^1 = f(\sum_{i=1}^{n+1} w_{ij}^i x_i), \text{ for } j = 1, .., n_{h^1} \tag{21}$$

$$H_j^2 = f(\sum_{i=1}^{n+1} w_{ij}^{h^1} x_i), \text{ for } j = 1, .., n_{h^2} \tag{22}$$

$$O_j = g(\sum_{i=1}^{n+1} w_i^{h^2} x_i) \tag{23}$$

where f and g are the Sigmoid and ReLU functions respectively, ω_{ij} and x_i are the respective layer weights and inputs. The training is carried for 1000 epochs using a data-set of 6400 samples.

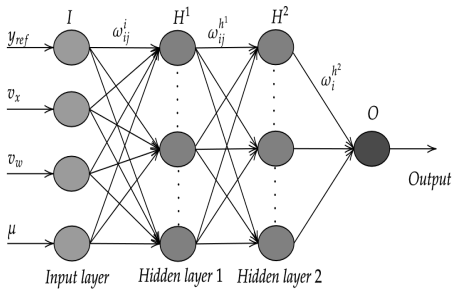

Fig. 3: Neural network for MPC adaptation

B. MPC adaptation with ANFIS

ANFIS tool is used in the second approach to learn the data-set of MPC optimal parameters in order to achieve online adaptation. Adaptive neuro-fuzzy inference systems are a hybrid AI technique that combines fuzzy logic and artificial neural networks to learn a mapping from input-output data [18]. The general architecture of an ANFIS system, illustrated in Fig. 4, is composed of the following:

- Layer 1: the fuzzification layer obtains fuzzy clusters from the input data by using membership functions as given below:
$$O_i^1 = \mu_{A_i}(x) \tag{24}$$
with x being the input to node i, and A_i the linguistic label of the node function. O_i^1 is the membership function.
- Layer 2: the rule layer computes the strengths by multiplying the membership values coming from the fuzzification layer:
$$\omega_i = \mu_{A_i}(x) \times \mu_{B_i}(y) \tag{25}$$
- Layer 3: the normalization layer computes the normalized strengths that belong to each rule by applying the following:
$$\bar{\omega_i} = \frac{\omega_i}{\sum_i^n \omega_i} \tag{26}$$
where $\bar{\omega_i}$ is the normalized strength and n is the number of nodes.
- Layer 4: the diffuzification layer calculates the weighted values of rules through first order polynomials :
$$\bar{\omega_i} f_i = \bar{\omega_i}(p_i x + q_i y + r_i) \tag{27}$$
with f_i being the polynomial composed of the parameter set $\{p_i, q_i, r_i\}$ called consequence parameters.
- Layer 5: the summation layer sums all the outputs of the diffuzification layer to obtain the ANFIS output:
$$O_i^5 = \frac{\sum_i \omega_i f_i}{\sum_i \omega_i} \tag{28}$$

The adaptation system consists of four ANFIS subsystems with four inputs each and one output corresponding to N_p, N_c, Q or R. The hybrid training approach combining gradient decent with RLS is used to train the ANFIS and avoid local minima. The scattering method is used for input

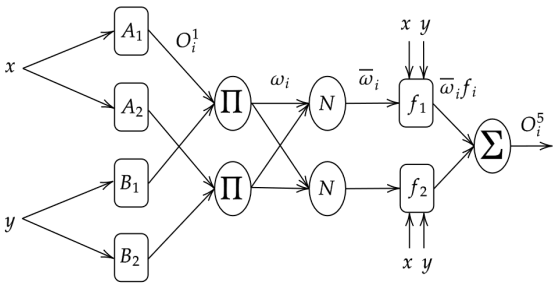

Fig. 4: General architecture of ANFIS

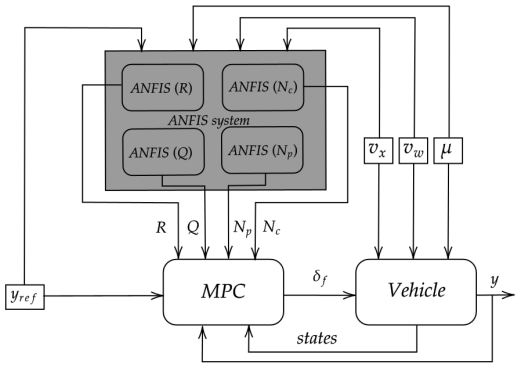

Fig. 5: ANFIS based adaptation approach

space partitioning since there are four inputs. Fig. 5 illustrates the proposed approach where v_x, v_w, μ and y_{ref} are the longitudinal velocity, the lateral wind velocity, the road adhesion coefficient and the reference lateral position.

V. Results and Discussion

For evaluating the proposed adaptive MPC controllers, a high fidelity vehicle model has been built using Vehicle Dynamics Blockset of MATLAB. The model consists of the 3-DOF dual track lateral dynamics block with nonlinear Pacejka tire formula [19]. In addition, a simplified powertrain block is added to accommodate variable velocities through the predictive longitudinal driver block. The parameters of model (7) and MPC controller are given in table (II).

Both NN-MPC and ANFIS-MPC are tested against the standard MPC for a triple lane change scenario which is often performed in multi-lane highways. The longitudinal velocity of the vehicle is varied according to Fig. 6(a). The vehicle was exposed to wind gust and varying road adhesion coefficient, these are given in Fig. 7 along with the resulting adaptive Nc, Np, Q and R signals. The corresponding trajectory tracking, steering angle, tracking error and yaw rate signals are given in Fig. 8. As can be seen from the figures, NN-MPC has the best tracking performance with an MSE of (0.0051) compared to (0.0062) and (0.0318) for the ANFIS-MPC and standard MPC respectively. On the other hand, ANFIS-MPC exhibits smoother control signals but is slightly less accurate. Overall, both adaptive controllers overcome the standard MPC and are much more robust and adaptive to wind disturbance and changing road conditions.

Furthermore, the controllers are tested for a general trajectory where the velocity profile is given in Fig. 6(b). Wind gust, road adhesion coefficient and Nc, Np, Q and R signals

TABLE II: MPC and model parameters

Model parameter	Value	MPC parameter	Value
m	1575 (kg)	N_p	35
I_z	$2875(kg.m^2)$	N_c	8
l_f	$1.2(m)$	$\Delta u_{max/min}$	$\pm\frac{\pi}{12}(rad)$
l_r	$1.6(m)$	$u_{max/min}$	$\pm\frac{\pi}{6}(rad)$
C_f	$19000(N/rad)$	R	0.01
C_r	$33000(N/rad)$	Q_y	10

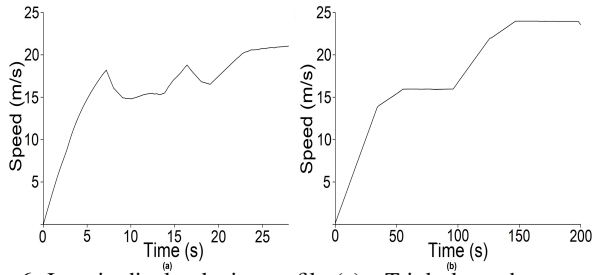

Fig. 6: Longitudinal velocity profile (a) : Triple lane change, (b) : General trajectory.

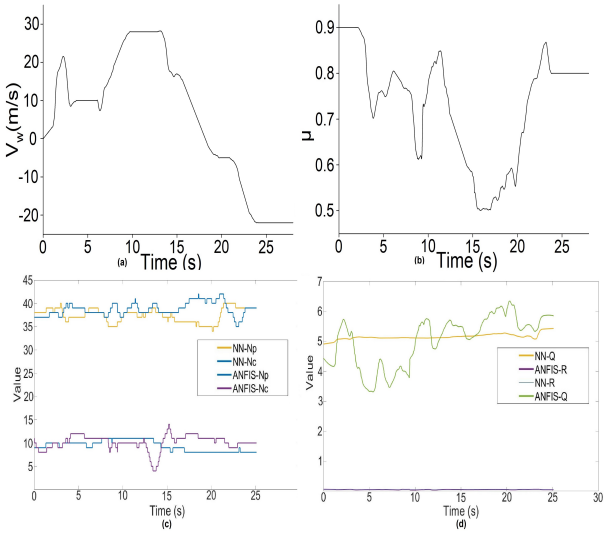

Fig. 7: (a) : Lateral gust profile, (b) : Road adhesion coefficient, (c) : N_c/N_p adaptation, (d) : Q/R adaptation.

are shown in Fig. 9 respectively. The corresponding results in Fig. 10 show similar performance to the previous test. The NN-MPC and ANFIS-MPC proved higher tracking accuracy, better disturbance rejection and are more adaptive compared to regular MPC. Similarly NN-MPC was slightly more accurate in this test with an MSE of (0.132) compared to (0.1349) and (0.5896) for ANFIS-MPC and classic MPC respectively. Contrary to NN-MPC, the main drawback of ANFIS-MPC is the curse of dimensionality. The latter is due to the exploding number of input membership functions, which makes such an approach very demanding in terms of computational power and less practical for real-time applications.

VI. Conclusions

In this paper, two adaptive MPC controllers are proposed for the lateral control of autonomous vehicles. An improved PSO algorithm is used to tune and optimize MPC parameters for varying working conditions and external disturbances. A data-set of optimal MPC parameters is generated and learned using two approaches; first by using MLP neural networks, and second by using ANFIS systems. The neural networks and ANFIS systems are then used for the adaptation of MPC parameters. The proposed controllers are tested against the standard MPC for a triple lane change scenario and a general trajectory. Furthermore, lateral gust is introduced with varying road conditions. The proposed

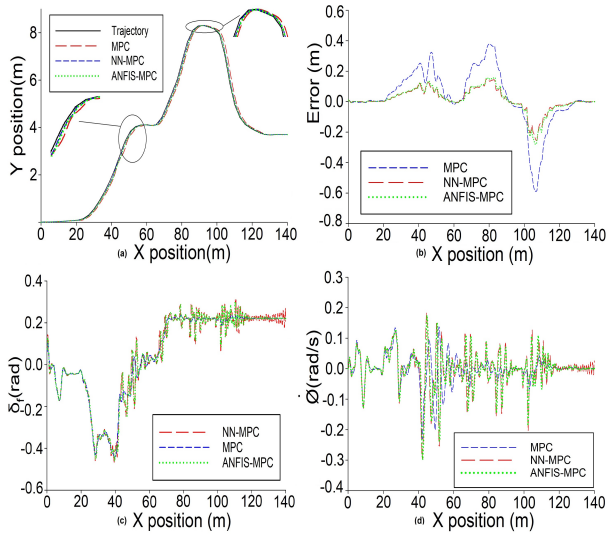

Fig. 8: Triple lane change ((a) : Trajectory tracking, (b) : Tracking error, (c) : Steering command, (d) : Vehicle yaw rate).

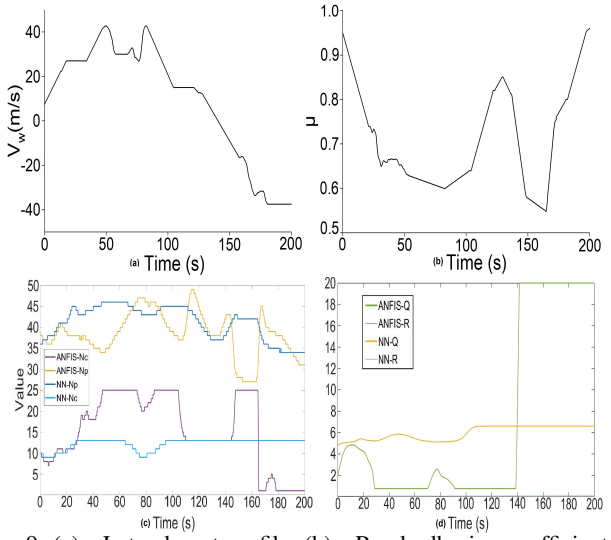

Fig. 9: (a) : Lateral gust profile, (b) : Road adhesion coefficient, (c) : N_c/N_p adaptation, (d) : Q/R adaptation.

controllers showed improved tracking accuracy, robustness and adaptability to disturbances and varying parameters. NN-MPC proved to be more accurate while ANFIS-MPC was found to be smooth but very demanding. Further studies shall address the improvement and learning of the MPC prediction model in the mixed longitudinal and lateral control.

REFERENCES

[1] V. Turri, A. Carvalho, H. E. Tseng, K. H. Johansson, and F. Borrelli, "Linear model predictive control for lane keeping and obstacle avoidance on low curvature roads," IEEE Conf. Intell. Transp. Syst. Proceedings, ITSC, no. Itsc, pp. 378–383, 2013

[2] P. F. Lima, M. Nilsson, M. Trincavelli, J. Martensson, and B. Wahlberg, "Spatial Model Predictive Control for Smooth and Accurate Steering of an Autonomous Truck," IEEE Trans. Intell. Veh., vol. 2, no. 4, pp. 238–250, 2017

[3] Y. Kebbati, N. Ait-Oufroukh, V. Vigneron, D. Ichalal, and D. Gruyer,"Optimized self-adaptive pid speed control for autonomous vehicles,"in2021 26th International Conference on Automation and Computing(ICAC), pp. 1–6, IEEE, 2021.

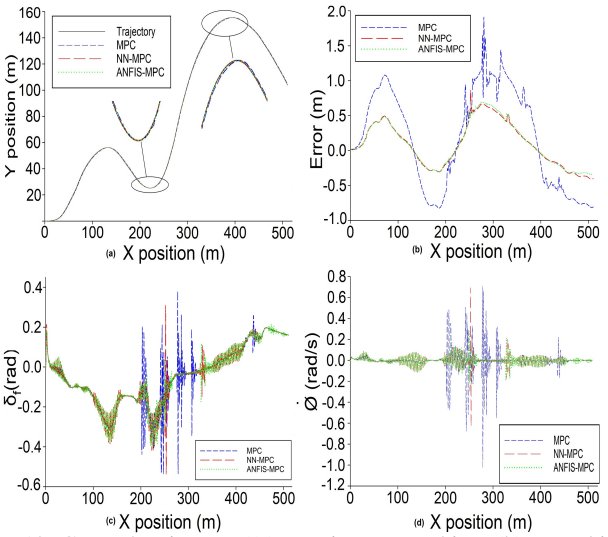

Fig. 10: General trajectory ((a) : Trajectory tracking, (b) : Tracking error, (c) : Steering command, (d) : Vehicle yaw rate).

[4] G. Han, W. Fu, W. Wang, and Z. Wu, "The lateral tracking control for the intelligent vehicle based on adaptive PID neural network," Sensors (Switzerland), vol. 17, no. 6, pp. 1–15, 2017

[5] M. Kissai, B. Monsuez, X. Mouton, D. Martinez, and A. Tapus, "Adaptive robust vehicle motion control for future over-actuated vehicles," Machines, vol. 7, no. 2, pp. 1–31, 2019

[6] E. Alcala, V. Puig, J. Quevedo, and T. Escobet, "Gain-scheduling LPV control for autonomous vehicles including friction force estimation and compensation mechanism," IET Control Theory Appl., vol. 12, no. 12, pp. 1683–1693, 2018

[7] E. Alcala, O. Sename, V. Puig, and J. Quevedo, "TS-MPC for Autonomous Vehicle using a Learning Approach," IFAC-PapersOnLine, vol. 53, no. 2, pp. 15110–15115, 2020

[8] M. Bujarbaruah, X. Zhang, H. E. Tseng, and F. Borrelli, "Adaptive MPC for Autonomous Lane Keeping," 2018, [Online]. Available: http://arxiv.org/abs/1806.04335.

[9] F. Lin, Y. Chen, Y. Zhao, and S. Wang, "Path tracking of autonomous vehicle based on adaptive model predictive control," Int. J. Adv. Robot. Syst., vol. 16, no. 5, pp. 1–12, 2019

[10] M. Brunner, U. Rosolia, J. Gonzales, and F. Borrelli, "Repetitive learning model predictive control: An autonomous racing example," 2017 IEEE 56th Annu. Conf. Decis. Control. CDC 2017, vol. 2018-January, no. Cdc, pp. 2545–2550, 2018

[11] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-Based Model Predictive Control for Autonomous Racing," IEEE Robot. Autom. Lett., vol. 4, no. 4, pp. 3363–3370, 2019

[12] B. Zhang, C. Zong, G. Chen, and B. Zhang, "Electrical Vehicle Path Tracking Based Model Predictive Control with a Laguerre Function and Exponential Weight," IEEE Access, vol. 7, pp. 17082–17097, 2019

[13] H. Wang, B. Liu, X. Ping, and Q. An, "Path Tracking Control for Autonomous Vehicles Based on an Improved MPC," IEEE Access, vol. 7, pp. 161064–161073, 2019

[14] Y. Kebbati, V. Puig, N. Ait-oufroukh, V. Vigneron and D. Ichalal, "Optimized adaptive MPC for lateral control of autonomous vehicles," 2021 9th Int. Conf. Control. Mechatronics Autom. ICCMA 2021.

[15] L. Wang,Model Predictive Control System Design and Implementation Using MATLAB, vol. 53. 2016.

[16] B. Song, Z. Wang, and L. Zou, "An improved PSO algorithm for smooth path planning of mobile robots using continuous high-degree Bezier curve," Appl. Soft Comput., vol. 100, p. 106960, 2021

[17] Y. Xie and W. Zhang, "A novel tuning method for PID controller based on improved PSO algorithm for unstable plants with time delay," Chinese Control Conf. CCC, vol. 2019-July, pp. 4270–4275, 2019

[18] J. R. Jang, "ANFIS : Adaptive-Network-Based Fuzzy Inference System," vol. 23, no. 3, 1993.

[19] H. B. Pacejka, (2008). Vehicle System Dynamics : International Journal of Vehicle Mechanics and Mobility. International Journal of Vehicle Mechanics and Mobility, (August 2012), 37–41